

We next define any constants for our program. First we define the RGB value for `BLACK`, which we will use to paint the background of our window. Then we define constants for the width and height of our window in pixels and a constant for the refresh rate for our program. This number defines the maximum number of times the program will loop (and therefore redraw the window) per second. Our value of 30 is fairly typical. If the amount of work done in our main loop is excessive, the program might run slower than this value, but it will never run faster. A refresh rate that's too high might cause the program to run too fast. In our ball example, this means the ball might bounce around the window faster than intended.

3. Initialize the pygame environment.

In this section, we call a function that tells pygame to initialize itself. We then ask pygame to create a window for our program with the `pygame.display.set_mode()` function and pass in the desired width and height of the window. Finally, we call another pygame function to create a clock object, which will be used at the bottom of our main loop to maintain our maximum frame rate.

4. Load assets: image(s), sound(s), and so on.

This is a placeholder section, into which we will eventually add code to load external images, sounds, and so on from the disk for use in our program. In this basic program we're not using any external assets, so this section is empty for now.

5. Initialize variables.

Here we will eventually initialize any variables that our program will use. Currently we have none, so we have no code here.

6. Loop forever.

Here we start our main loop. This is a simple `while True` infinite loop. Again, you can think of each iteration through the main loop as one frame in an animation.

7. Check for and handle events; commonly referred to as the *event loop*.

In this section, we call `pygame.event.get()` to get a list of the events that happened since the last time we checked (the last time the main loop